



4강. Git과 협업 워크플로우



학습 목표

- 버전 관리가 왜 필요한지, Git의 분산 구조가 무엇인지 설명할 수 있습니다.
- Git의 4개 공간(작업 디렉터리·스테이징·로컬 저장소·원격 저장소)과 데이터 이동 명령을 사용할 수 있습니다.
- 브랜치를 만들어 작업하고, merge와 rebase의 차이를 이력 관점에서 설명하고 상황에 맞게 선택할 수 있습니다.
- 충돌(conflict)이 왜 발생하는지 이해하고 직접 해결할 수 있습니다.
- Pull Request 기반 협업 사이클(브랜치 → 커밋 → 푸시 → PR → 리뷰 → 병합)을 수행할 수 있습니다.



목차

들어가며

1. Git은 어떻게 동작할까요? — 스냅샷과 분산 구조
2. Git의 4개 공간
3. 브랜치 — 서로 방해하지 않는 평행 우주
4. 합치기 — merge vs rebase
5. 충돌(conflict) — 두려워할 것 없습니다
6. Pull Request — 리뷰를 통과해야 main에 들어간다

미니 퀴즈

실습

정리

들어가며

로봇 프로젝트의 흔한 참사 시나리오입니다. 팀원 A가 LiDAR 드라이버를 고치는 동안 팀원 B가 같은 파일의 모터 제어 코드를 수정했습니다. B가 파일을 덮어쓰는 순간 A의 하루치 작업이 사라졌습니다. 더 나쁜 경우 — "지난주에는 로봇이 잘 돌아갔는데, 누가 뭘 바꿨는지 몰라서 되돌릴 수가 없다."

이 문제를 해결하는 도구가 **버전 관리 시스템(VCS)**, 그중 사실상 표준인 **Git**입니다. Git은 코드의 모든 변경 이력을 **스냅샷**으로 남기고, 여러 사람이 **동시에, 서로를 덮어쓰지 않고** 작업하게 해 줍니다. ROS2 생태계의 모든 패키지가 GitHub 위에서 개발되고 있으므로, Git은 로봇 개발자의 필수 기본기입니다.

1. Git은 어떻게 동작할까요? — 스냅샷과 분산 구조

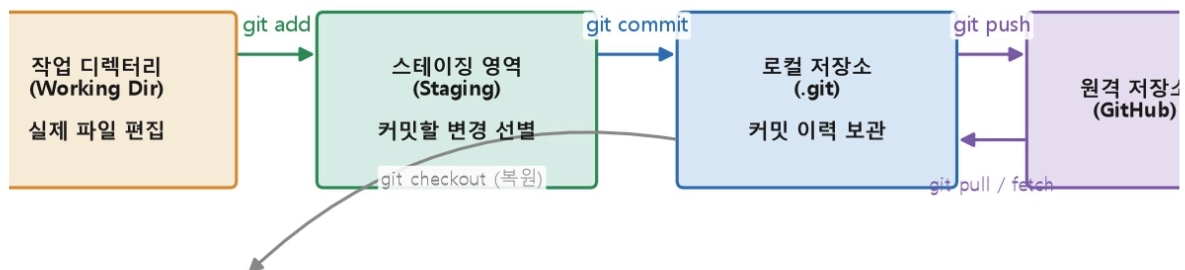
Git의 핵심 개념 두 가지를 먼저 잡겠습니다.

- **스냅샷(snapshot)**: Git은 "무엇이 바뀌었나(차이)"가 아니라 "**그 시점 프로젝트 전체의 모습**"을 저장합니다. 각 스냅샷이 **커밋(commit)**이며, 고유한 해시 ID(예: `a3f9c2e`)를 갖습니다. 커밋들은 부모를 가리키는 사슬로 연결되어 **이력(history)**이 됩니다.
- **분산(distributed)**: 모든 개발자의 PC에 **저장소 전체 이력의 완전한 복사본**이 있습니다. 서버(GitHub)가 죽어도 누군가의 로컬에 전체 이력이 남아 있고, 네트워크 없이도 커밋·이력조회·브랜치가 다 됩니다. 로봇 현장(네트워크가 불안정한 공장·야외)에서 특히 고마운 특성입니다.

2. Git의 4개 공간

Git을 처음 배울 때 가장 헷갈리는 것이 "내 변경이 지금 어디에 있는가"입니다. 공간을 4개로 나누면 명확해집니다.

git의 4개 공간과 데이터의 이동



commit = 스냅샷 저장, push/pull = 원격과 동기화

💡 **해석:** 파일 편집은 **작업 디렉터리**에서 일어납니다. `git add` 로 커밋할 변경만 **스테이징**에 골라 담고, `git commit` 으로 **로컬 저장소**에 스냅샷을 기록합니다. `git push` 로 **원격 저장소(GitHub)**에 올려 팀과 공유하고, 팀의 변경은 `git pull` 로 받아옵니다. 스테이징이 중간에 있는 이유는 "**수정한 것 중 일부만 골라 의미 있는 단위로 커밋**" 하기 위해서입니다.

기본 흐름을 명령으로 보면:

```
git clone https://github.com/team/robot-ws.git # 원격 저장소 복제 (최초 1회)
cd robot-ws # ... 파일 수정 ...
git status # 어떤 파일이 바뀌었는지 확인 (습관!)
git diff # 구체적으로 무엇이 바뀌었는지 확인
git add src/lidar_driver.py # 커밋할 변경 선택
git commit -m "LiDAR 타임아웃 5초로 연장" # 스냅샷 기록
git push origin main # 원격에 공유
```

커밋 메시지는 미래의 팀원(그리고 미래의 나)에게 보내는 기록입니다. "**무엇을, 왜**"가 담기게 씁니다. `git log --oneline --graph`로 이력을 그래프로 확인하는 습관을 들이세요.

3. 브랜치 — 서로 방해하지 않는 평행 우주

브랜치(branch)는 이력에서 갈라져 나온 독립적인 작업 흐름입니다. 원리는 단순합니다 — 브랜치는 사실 **특정 커밋을 가리키는 포인터**일 뿐이라, 만드는 데 비용이 거의 들지 않습니다.

```
git switch -c feature/lidar-driver # 브랜치 생성 + 이동 # ... 작업, 커밋 반복 ...
git switch main # 언제나 main으로 복귀 (작업물은 브랜치에 안전)
```

브랜치를 쓰는 규칙은 명확합니다.

💡 **main 브랜치는 "항상 동작하는 코드"만 둔다.** 모든 작업은 브랜치에서 하고, 검증을 거쳐 main에 합친다.

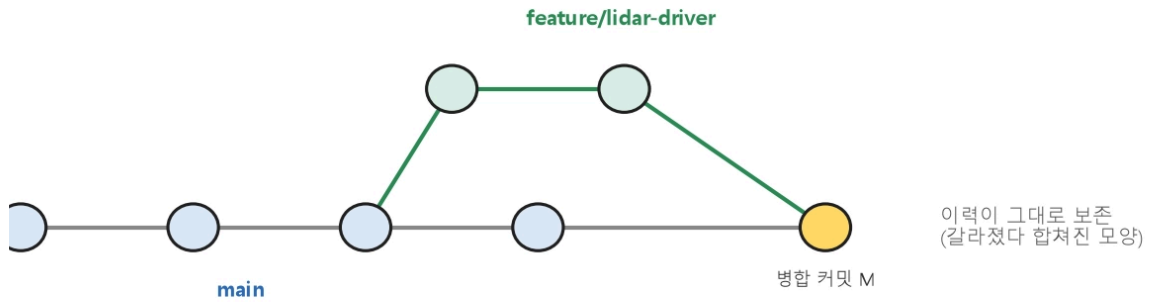
로봇 프로젝트라면 "main = 지금 로봇에 올려도 되는 코드"입니다. 실험적인 제어 게인 튜닝, 새 센서 드라이버는 전부 `feature/...` 브랜치에서 진행합니다.

4. 합치기 — merge vs rebase

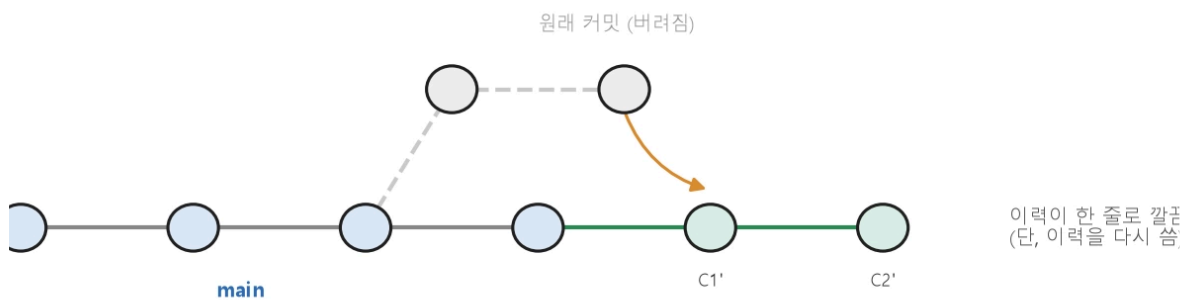
브랜치에서 작업을 마치면 main에 **합쳐야** 합니다. 방법이 두 가지인데, 결과 코드는 같아도 **이력의 모양**이 달라집니다.

merge vs rebase — 같은 목표, 다른 이력

두 이력을 합치는 '병합 커밋'을 만든다



내 커밋을 최신 main 위로 ' 옮겨 다시 쌓는다'



images/04_merge_vs_rebase.png — merge vs rebase

💡 **해석:** (위) **merge**는 두 갈래를 합치는 **병합 커밋 M**을 새로 만듭니다. "언제 갈라졌고 언제 합쳐졌는지" 이력이 사실 그대로 보존됩니다. (아래) **rebase**는 내 커밋들(C1, C2)을 최신 main 끝으로 **옮겨 다시 쌓아**(C1', C2') 이력을 한 줄로 만듭니다. 깔끔하지만, 원래 커밋을 버리고 **새 커밋으로 다시 쓰는** 작업입니다.

```
# merge 방식 git switch main git merge feature/lidar-driver #
rebase 방식 (브랜치를 최신 main 위로 재배치 후 병합) git switch
feature/lidar-driver git rebase main git switch main git merge
feature/lidar-driver # 한 줄이 되어 fast-forward 병합
```

어떤 걸 쓸까요? 실무의 일반적인 합의는 이렇습니다.

상황	선택	이유
공유 브랜치(main)에 기능 합치기	merge (PR 병합)	이력 보존, 협업 기록 명확
내 로컬 브랜치를 최신 main과 동기화	rebase	지저분한 중간 병합 커밋 없이 깔끔
이미 push한(공유된) 커밋	rebase 금지	이력을 다시 쓰면 팀원들의 이력과 충돌

- ❗ 반드시 기억하세요: 원격에 push해서 다른 사람이 봤을 수 있는 커밋은 **rebase하지 않습니다**. rebase는 커밋을 "새로 다시 쓰는" 것이므로, 공유된 이력을 rebase하면 팀 전체의 저장소가 꼬입니다. rebase는 **아직 나만 가진 로컬 커밋**에만 씁니다.

5. 충돌(conflict) — 두려워할 것 없습니다

merge든 rebase든, **같은 파일의 같은 부분**을 양쪽에서 다르게 고쳤다면 Git은 어느 쪽이 옳은지 판단할 수 없어 **충돌**을 선언하고 사람에게 넘깁니다. 충돌은 오류가 아니라 **"여기는 사람이 정해 주세요"**라는 **정상적인 질문**입니다.

충돌이 나면 파일 안에 표식이 삽입됩니다:

```
def get_max_speed(): <<<<<<< HEAD return 0.8 # 안전을 위해 감속 (main
의 변경) ===== return 1.2 # 성능 테스트용 증속 (내 브랜치의 변경)
>>>>>>> feature/speed-tuning
```

해결 절차는 세 단계입니다.

1. <<<<<<<, =====, >>>>>>> 사이의 두 버전을 읽고, **올바른 최종 코드를 직접 작성**합니다(둘 중 하나 선택, 또는 둘을 조합). 표식은 모두 지웁니다.
2. `git add 파일명` 으로 "해결했음"을 표시합니다.
3. `git commit` (merge 중) 또는 `git rebase --continue` (rebase 중)로 마무리합니다.

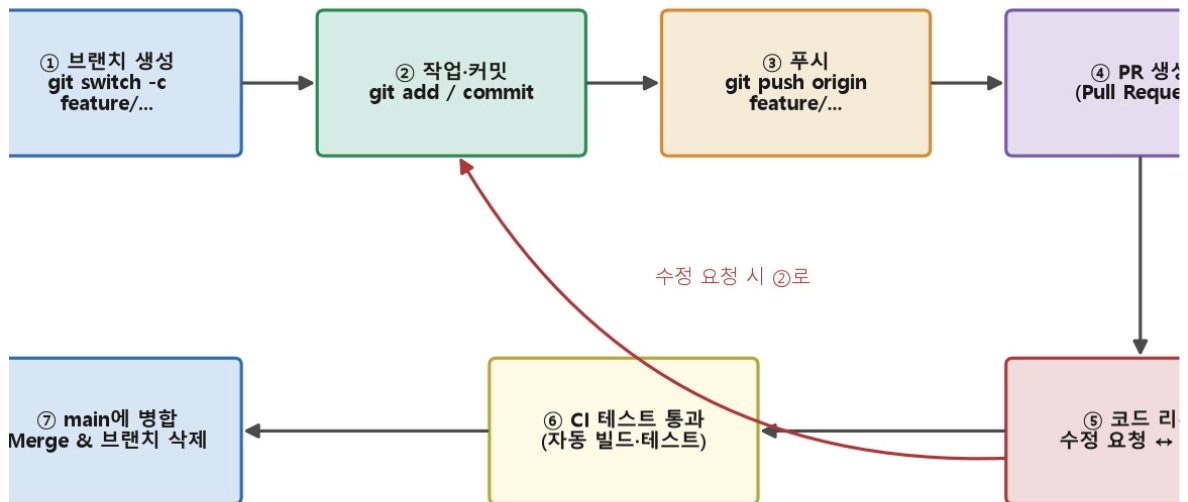
위 예시라면 "안전 한도는 0.8로 유지하되 테스트 플래그로 1.2 허용" 같은 **제3의 올바른 코드**를 쓰는 것이 정답일 수 있습니다. 충돌 해결은 기계적 선택이 아니라 **코드 의도의 통합**입니다.

충돌을 줄이는 예방책도 있습니다 — 브랜치를 **작게, 짧게** 유지하고(하루~며칠 단위), main의 변경을 **자주 가져와**(`git pull` 후 rebase) 격차를 줄이는 것입니다.

6. Pull Request — 리뷰를 통과해야 main에 들어간다

여러 명이 협업할 때, 브랜치를 main에 합치는 관문이 **Pull Request(PR)**입니다. "내 브랜치를 main에 당겨가(pull) 주세요"라는 **요청 + 리뷰의 장**입니다.

Pull Request 협업 사이클



main 브랜치는 항상 '동작하는 코드'로 보호되고, 모든 변경은 리뷰와 테스트를 거쳐 들어갑니다

images/04_PR_사이클.png — Pull Request 협업 사이클

💡 **해석:** ① 브랜치 생성 → ② 작업·커밋 → ③ 푸시 → ④ PR 생성 → ⑤ 코드 리뷰(수정 요청이 오면 ②로 돌아가 커밋을 추가) → ⑥ CI 자동 테스트 통과 → ⑦ main 병합 및 브랜치 삭제. **main은 리뷰와 테스트를 통과한 코드만 받는 보호 구역이 됩니다.**

리뷰가 형식적인 절차가 아닌 이유:

- **결함 조기 발견:** 로봇 코드의 버그는 물리적 사고가 됩니다. 두 번째 눈이 지나가는 것만으로 결함 검출률이 크게 올라갑니다.
- **지식 공유:** 리뷰를 통해 팀 전체가 코드베이스를 이해하게 됩니다. "그 부분은 A만 안다"는 위험이 줄어듭니다.
- **컨벤션 유지:** 코드 스타일·설계 원칙이 자연스럽게 전파됩니다.

리뷰를 주고받는 요령 — 리뷰어는 코드가 아니라 **코드의 문제**를 지적하고(사람 비난 금지), 작성자는 방어 대신 **의도를 설명**합니다. PR은 작게(리뷰 가능한 크기, 수백 줄 이내) 유지하는 것이 서로를 돕는 길입니다.

- ▶ 심화 — CI(지속적 통합)와 로봇 프로젝트
- ▶ 심화 — 되돌리기 도구들: restore, revert, reset

미니 퀴즈

💡 아래 문항을 먼저 스스로 풀어 본 뒤 실습으로 넘어가세요. 답안은 실습 결과물과 함께 제출합니다. (총 10점)

1. (2점) Git의 4개 공간 중, `git add` 로 파일이 이동하는 곳은?

- ① 작업 디렉터리
- ② 스테이징 영역
- ③ 로컬 저장소
- ④ 원격 저장소

2. (2점) 이미 push 한 커밋에 rebase 를 쓰면 안 되는 이유를 설명하세요.

3. (2점) 공유된 main 에 잘못된 커밋이 들어갔습니다. 이력을 보존하면서 되돌리는 안전한 명령은?

- ① `git reset --hard`
- ② `git revert 커밋ID`
- ③ `git rebase -i`
- ④ `git checkout --`

4. (2점) Pull Request를 쓰는 이유를 두 가지 쓰세요.

5. (2점) `.gitignore` 에 넣어야 할 것과 넣으면 안 되는 것의 예를 하나씩 쓰세요.

실습

혼자서도 전체 협업 사이클을 연습할 수 있습니다. GitHub에 연습용 저장소를 만들고 다음을 수행해 보겠습니다.

- 기본 사이클:** 저장소를 clone → 파일 수정 → `status` / `diff` 확인 → add → commit → push까지 수행하고, GitHub에서 커밋이 보이는지 확인해 보세요.
- 브랜치와 PR:** `feature/readme-update` 브랜치를 만들어 README를 수정·푸시하고, GitHub에서 PR을 열어 스스로 리뷰 코멘트를 단 뒤 병합해 보세요.
- 충돌 만들기과 해결:** 브랜치 두 개(`branch-a` , `branch-b`)에서 **같은 줄**을 서로 다르게 수정하고 차례로 병합해, 의도적으로 충돌을 발생시킨 뒤 해결해 보세요. 해결 과정에서 `<<<<<<<` 표식이 무엇을 의미했는지 설명을 적어 보세요.
- (심화) rebase 체험:** `branch-b` 를 병합하기 전에 `git rebase main` 으로 최신화하고, `git log --oneline --graph` 로 merge 방식과 이력 모양이 어떻게 다른지 비교해 보세요.

정리

- Git은 프로젝트의 **스냅샷(커밋)** 사슬로 이력을 관리하는 **분산** 버전 관리 시스템입니다.
- 변경은 **작업 디렉터리** → **(add)** → **스테이징** → **(commit)** → **로컬 저장소** → **(push)** → **원격** 순으로 이동합니다.
- **main**은 **항상 동작하는 코드로** 보호하고, 작업은 브랜치에서 합니다.
- **merge**는 이력 보존, **rebase**는 이력 정리 — 단, 공유된 커밋은 절대 **rebase**하지 않습니다.
- 충돌은 오류가 아니라 "사람이 정해 달라"는 질문이며, 두 변경의 **의도를 통합**해 해결합니다.